

CS 300 Project One: Pseudocode and Runtime Analysis

Course Structure

Define Course

courseNumber

courseName

prerequisites as a list of strings

Vector Pseudocode

Function loadCoursesVector(fileName)

Create empty vector called courses

Open file

If file cannot be opened

Print error message

Return empty vector

For each line in file

Split line by commas into tokens

If tokens has fewer than 2 items

Print file format error

Return empty vector

Create Course object

Set course.courseNumber = tokens[0]

Set course.courseName = tokens[1]

For each token after tokens[1]

 Add token to course.prerequisites

Add course to courses vector

For each course in courses

 For each prerequisite in course.prerequisites

 If prerequisite does not exist as a courseNumber in courses

 Print prerequisite error

 Return empty vector

Return courses

Function printCourseListVector(courses)

 Sort courses alphanumerically by courseNumber

 For each course in courses

 Print course.courseNumber and course.courseName

Function searchCourseVector(courses, courseNumber)

 For each course in courses

 If course.courseNumber equals courseNumber

Print course.courseNumber and course.courseName

Print prerequisites or "Prerequisites: None"

Return

Print "Course not found"

Hash Table Pseudocode

Function loadCoursesHash(fileName)

Create empty hash table called courses

Create empty set called courseNumbers

Open file

If file cannot be opened

Print error message

Return empty hash table

For each line in file

 Split line by commas into tokens

 If tokens has fewer than 2 items

 Print file format error

 Return empty hash table

 Add tokens[0] to courseNumbers

Close file and reopen file

For each line in file

Split line by commas into tokens

Create Course object

Set course.courseNumber = tokens[0]

Set course.courseName = tokens[1]

For each token after tokens[1]

If prerequisite does not exist in courseNumbers

Print prerequisite error

Return empty hash table

Add prerequisite to course.prerequisites

Insert course into hash table using course.courseNumber as key

Return courses

Function printCourseListHash(courses)

Create temporary vector called sortedCourses

For each bucket in hash table

Add each course in bucket and chained nodes to sortedCourses

Sort sortedCourses alphanumerically by courseNumber

For each course in sortedCourses

Print course.courseNumber and course.courseName

Function searchCourseHash(courses, courseNumber)

Search hash table using courseNumber

If course is found

Print course.courseNumber and course.courseName

Print prerequisites or “Prerequisites: None”

Else

Print “Course not found”

Binary Search Tree Pseudocode

Function loadCoursesTree(fileName)

Create empty Binary Search Tree called courses

Create empty set called courseNumbers

Open file

If file cannot be opened

Print error message

Return empty tree

For each line in file

Split line by commas into tokens

If tokens has fewer than 2 items

Print file format error

Return empty tree

Add tokens[0] to courseNumbers

Close file and reopen file

For each line in file

Split line by commas into tokens

Create Course object

Set course.courseNumber = tokens[0]

Set course.courseName = tokens[1]

For each token after tokens[1]

If prerequisite does not exist in courseNumbers

Print prerequisite error

Return empty tree

Add prerequisite to course.prerequisites

Insert course into Binary Search Tree by courseNumber

Return courses

Function printCourseListTree(node)

If node is not null

Call printCourseListTree(node.leftChild)

Print node.course.courseNumber and node.course.courseName

Call printCourseListTree(node.rightChild)

Function searchCourseTree(tree, courseNumber)

Set current node to root

While current node is not null

If current course number equals courseNumber

Print course number and course name

Print prerequisites or “Prerequisites: None”

Return

Else if courseNumber is less than current course number

Move to left child

Else

Move to right child

Print “Course not found”

Menu Pseudocode

Display menu

1. Load Course Data

2. Print Course List

3. Print Course Information

9. Exit

While user choice is not 9

Get user choice

If choice is 1

Load file data into selected data structure

If choice is 2

Print alphanumerically sorted course list

If choice is 3

Ask user for course number

Print course title and prerequisites

If choice is 9

Exit program

Runtime Analysis

Data Structure	Load/Create Courses	Search Course	Print Sorted Course List	Memory
Vector	$O(n^2)$, because prerequisite validation may require	$O(n)$	$O(n \log n)$ if sorting is needed	$O(n)$

	searching all courses			
Hash Table	$O(n)$, average case, because insertion and lookup are constant time	$O(1)$, average case	$O(n \log n)$, because courses must be copied and sorted	$O(n)$
Binary Search Tree	$O(n \log n)$, average case, if reasonably balanced	$O(\log n)$, average case	$O(n)$, because in-order traversal prints sorted data	$O(n)$

Advantages and Disadvantages

A vector is simple to use and stores data in a straightforward way. It is easy to add courses and loop through them. However, searching for a specific course is slower because the program may need to check every course. Printing in alphanumeric order also requires sorting the vector.

A hash table is very efficient for searching because it can find a course quickly using the course number as the key. This makes it strong for printing information about one specific course.

However, a hash table does not naturally store courses in sorted order, so printing all courses alphanumerically requires copying the data into another structure and sorting it.

A binary search tree is useful because it organizes courses by course number as they are inserted. Searching is efficient when the tree is balanced, and an in-order traversal naturally prints the courses in alphanumeric order. A disadvantage is that if courses are inserted in already sorted order, the tree can become unbalanced and perform more like a linked list.

Recommendation

Based on the advisor's requirements, I recommend using a binary search tree. The program must print all computer science courses in alphanumeric order and also print information for a specific course. A binary search tree supports both needs well because it keeps course numbers ordered and can print the full list using in-order traversal in $O(n)$ time. It also provides efficient searching in $O(\log n)$ average time when the tree is reasonably balanced. While a hash table is faster for individual lookups, it does not naturally support sorted output. Since sorted course output is a major requirement, the binary search tree is the best overall choice for this project.